

Le temps c’est de l’argent …
et des bugs aussi !

Nos sponsors BDX I/O 2026

Merci à eux !

BALTHAZAR

Betclic

4SH catamania DEEZER ippon POSITIVE TECHNOLOGY néowsoft onepoint.

IMPÉRIALE

Peaksys Tech de CAISCOUNT [sfeir] sqli

MAGNUM

LECTRA niji zenika

STARTUP

gemster Graybox QueryVision imeon tok en cli mate zeno

BDX I/O



Julien Lenormand

#human #craft

Qui suis-je ?

Dev = j'écris du code

Software Craft = apprendre ensemble à faire bien

- gestion de personnel ferroviaire
- gestion de crise nucléaire
- appareils de mesure électrique
- suivi de travaux BTP
- gestion d'entrepôt logistique
- scanners à bagages aéroportuaires
- bornes de recharge de voiture électrique
- télémétrie de matériel médical
- analyse de trajets de camions routiers
- détection/surveillance de sites de phishing
- ...

Le temps

Un problème récurrent :

- toujours présent
- intrinsèque à toutes les activités humaines
- crucial dans de nombreux domaines
- si intuitif
- et pourtant si compliqué

Définition : le temps

> Milieu indéfini et homogène dans lequel se situent les êtres et les choses et qui est caractérisé par sa double nature, à la fois continuité et succession. — cntrl.fr

> Continuité indéfinie, milieu où se déroule la succession des évènements et des phénomènes, les changements, mouvements, et leur représentation dans la conscience. — dictionnaire.lerobert.com



Ce qu'en dit la philo :

- opérationaliste = c'est ce qu'on mesure, point
- substantialisme absolutiste = c'est compliqué

On va se passer de cette définition aujourd'hui …

SKIP

BDX I/O

Définition : les bugs (temporels)



BDX I/O

C’est parti pour un voyage …

A travers le(s) temps !



BDX I/O

Les problèmes commencent ...

- Dimanche 25 Octobre 2026, je rentre de soirée ...
- mon Apple Watch affiche : 02 h 30
- je marche 45 minutes
- j'arrive : 02 h 15 😰 avant d'être parti ?
- je vais me coucher : 02 h 30 de nouveau 😰 encore ?!

Que s'est-il passé ?



Changement d'heure : "daylight saving(s) time"

BDX I/O

Heure d’hiver, avec du code

```python
import datetime as dt

depart = dt.datetime.now()
# ... 45 minutes de marche ...
arrivee = dt.datetime.now()
# ... je me couche ...
coucher = dt.datetime.now()
print(arrivee - depart)
# -15 minutes
print(coucher == depart)
# True
```


BDX I/O

Changement d’heure : hiver



Problèmes du temps

- Non-unicité : DST
- Non-monotonicité : DST

Et dans l’autre sens

- Heure de livraison : 02 h 30 (du matin), le Dimanche 29 Mars 2026
- Durée du trajet : 45 minutes + 1 heure !!!
- Heure de départ : 01 h 45
- Heure d'arrivée : 03 h 30 😰

Que s'est-il passé ?
02 h 30 n'a pas existé ce jour-là : après 01 h 59, il était 03 h 00

Heure d’été, avec du code

```
import datetime as dt

start_time = dt.datetime(2026, 3, 29, 1, 45, 0)
end_time =   dt.datetime(2026, 3, 29, 3, 30, 0)
duration = end_time - start_time
print(duration)
# 1:45:00
```


BDX I/O

Changement d’heure : été



BDX I/O

Problèmes du temps

- Non-unicité : DST
- Non-monotonicité : DST
- Non-continuité : DST

Et encore dans l’autre sens



BDX I/O

Problèmes du temps

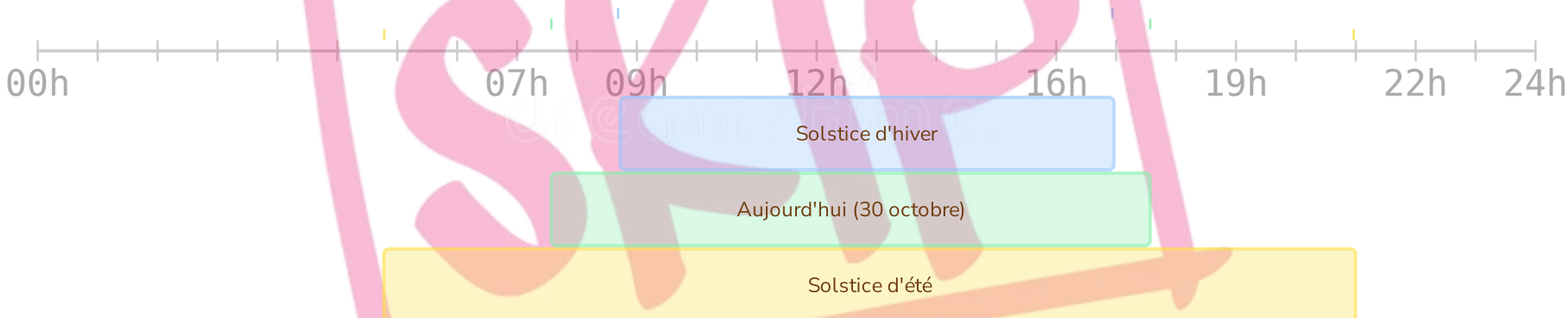
- Non-unicité : DST
- Non-monotonicité : DST
- Non-continuité : DST
- Non-homogénéité : DST

Pourquoi s’infliger ça ?

Réponse n°1 : 🛢️ 💲

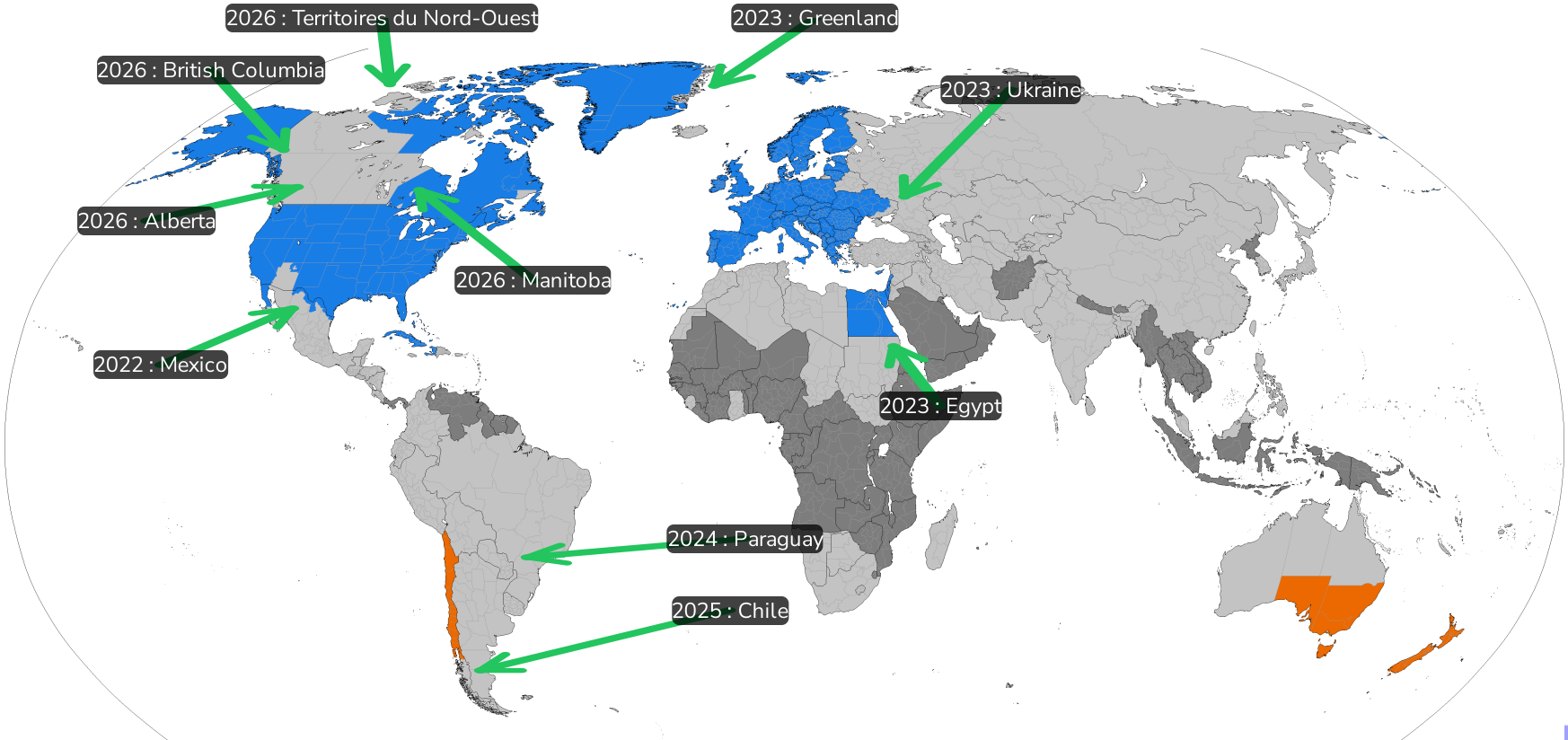
> Le changement d’heure a été instauré en France à la suite du choc pétrolier de 1973-1974. — service-public.gouv.fr

Réponse n°2 : 🌅 ☃️



BDX I/O

Évolutions



BDX I/O

Problèmes du temps

- Non-unicité : DST
- Non-monotonicité : DST
- Non-continuité : DST
- Non-homogénéité : DST
- Non-constance : DST

Éviter / détecter (DST) : soustraire deux datetimes

```python
import datetime as dt
from zoneinfo import ZoneInfo
paris = ZoneInfo("Europe/Paris")

premier = dt.datetime(2026, 10, 25, 2, 30, tzinfo=paris)  # fold=0
second  = dt.datetime(2026, 10, 25, 2, 30, tzinfo=paris, fold=1)
print(premier)  # 2026-10-25 02:30:00+02:00
print(second)   # 2026-10-25 02:30:00+01:00
print(second - premier)  # 0:00:00  (python/cpython#112638)
print(second.timestamp() - premier.timestamp())  # 3600.0
```

- Écrire des tests avec une horloge qu'on contrôle
  - `time-machine`, `freezegun`, `libfaketime`, Symfony Clock (ou `MockClock` en PHP), …

BDX I/O

Éviter / détecter (DST) : une durée se mesure en monotone

```python
# processus lancé avec TZ=Europe/Paris
import datetime as dt, time, time_machine
from zoneinfo import ZoneInfo
paris = ZoneInfo("Europe/Paris")
before = dt.datetime(2026, 10, 25, 2, 50, tzinfo=paris)         # 02:50 +02:00
after  = dt.datetime(2026, 10, 25, 2, 5, tzinfo=paris, fold=1)  # 02:05 +01:00
```

Horloge murale

```python
with time_machine.travel(before) as clock:
    start = dt.datetime.now()
    time.sleep(15 * 60)
    clock.move_to(after)
    delta = dt.datetime.now() - start
    print(delta.total_seconds())
# -2700.0
```

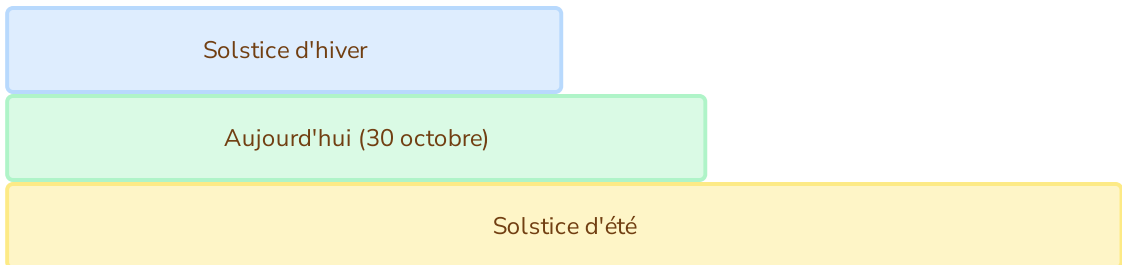
Horloge monotone

```python
with time_machine.travel(before) as clock:
    start = time.monotonic()
    time.sleep(15 * 60)
    clock.move_to(after)
    delta = time.monotonic() - start
    print(delta)
# 900.0006…
```

```python
print(round(time.monotonic(), 2))  # 24164.39
```


BDX I/O

Avez-vous remarqué ?



Le soleil, c’est compliqué

Vraiment très compliqué ...

```
today.replace(month=11)  # 30 novembre 2026
tomorrow.replace(month=11)  # ValueError: day 31 must be in range 1..30 for month 11 in year 2026
# marche aujourd'hui, mais pas demain
```

site Hugo grenoble-meetups.fr, bouton « mois suivant » : `$now.AddDate 0 1 0`

le 31 août → « 31 septembre » → 1er octobre

BDX I/O

La périodicité naturelle

- le jour ☀️, la nuit 🌌🌙
- les 4 saisons astronomiques : 🌱🌻🍂🌳
- les saisons équatoriales: 🌧️☀️
- les années 📆

BDX I/O



Oui, mais ...

- jour solaire ? civil ?
  - jour solaire : centré sur midi solaire (zénith)
  - aujourd'hui le 30 octobre : 7:32 (CET) --> 17:35 (CET)
  - donc midi solaire à (7:32+17:35)/2 = 12:33 (CET)
  - donc midi solaire à pas du tout "midi"
- année tropicale ? sidérale ? civile ?
  - tropicale (retour du soleil au même point dans le ciel) : ~365.24219 jours "éphémérides" (2000)
  - sidérale (retour de la Terre au même point sur son orbite) : 365.256363 jours (2025)
  - différence de ~20 minutes et 24.7 seconds
  - anodin ?

Une histoire de chocolat ?

- Jésus-Christ a ressuscité [citation needed]
  - les chrétiens ont souhaité commémorer cette date : Pâques
  - 1er dimanche après la pleine lune après le 21 Mars (approximation de l'équinoxe de Mars)
  - date mouvante = 😬
- le calendrier Julien supposait qu'une année faisait exactement 365.25 jours
  - année tropicale ≈ 365.2422
  - les dates dérivent : le 21 Mars ne colle plus à l'équinoxe !
  - 1545 : on souhaite revenir à la « vraie » date qu'il était en l'an 325
- calendrier Grégorien !
  - transition : 10 jours sautés d'un coup

JULIEN	SEP 30	OCT 1	OCT 2	OCT 3	OCT 4	OCT 5	OCT 6	OCT 7	OCT 8
GRÉGORIEN	OCT 10	OCT 11	OCT 12	OCT 13	OCT 14	OCT 15	OCT 16	OCT 17	OCT 18

  - année = 365.2425
  - année = 365 ou 366 jours ?

BDX I/O

Quiz sur les calendriers !

1. Règle pour les années bissextiles ❓
- divisible par 4
- mais **pas** par 100
- mais quand même par 400
2. Pourquoi "bissextile" ❓
- car dans le calendrier Julien il s'insérait après le 24 Février
- > ante diem bis sextum Kalendas Martias
- > le sixième jour bis avant les calendes (le premier jour) de mars

ℹ️ En anglais : "leap years"

3. Année civile basée sur : tropicale ou sidérale ❓
- tropicale
- on aime les équinoxes constants à travers les siècles (🤨)

BDX I/O

Problèmes du temps

- Non-unicité : DST
- Non-monotonicité : DST
- Non-continuité : DST
- Non-homogénéité : DST
- Non-constance : DST, leap years

Les constantes qui ne le sont pas

- ☪️ Musulman (Hijri / Hégirien / "islamic") :
  - lunaire ("basé sur la lune")
  - la lune n'a pas une période synodique de X jours exactement (~29.53)
  - année de 12 mois lunaires : 354 ou 355 jours
  - (6×29 + 6×30 = 354 ; 5×29 + 7×30 = 355)
  - les saisons tournent sur 33 ans
- ✡️ Juif (hébraïque / "hebrew") :
  - mois luni-solaires "intercalaires" pour approximer l'année tropicale
  - un mois supplémentaire tous les 2 ou 3 ans
  - cycle métonique : 7 mois en 19 ans
- ☦️ Orthodoxe :
  - resté sur le calendrier Julien
  - 13 jours de retard (non-rattrapés par le passage au Grégorien)
  - règle des bisextiles Julien : simplement divisible par 4
- calendriers 🕉️ bouddhiste, 🐉 chinois, 🕉️ hindou, ...

Le calendrier ✝️ grégorien est le standard dans l'industrie occidentale.

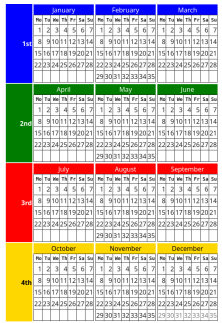
SKIP

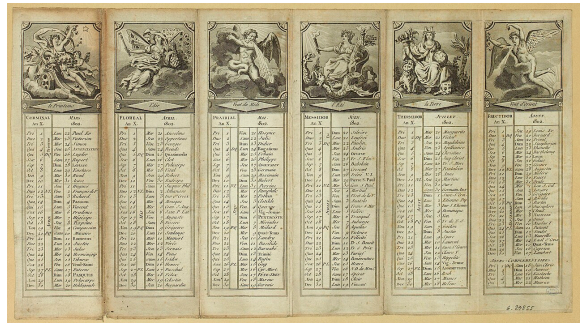
BDX I/O

Problèmes du temps

- Non-unicité : DST
- Non-monotonicité : DST
- Non-continuité : DST
- Non-homogénéité : DST, leap years
- Non-constance : DST, leap years

Solution (calendriers) : réformer le calendrier ?





Symmetry 454

Semaine intercalaire !! 😭

(52 × année + 146) mod 293 < 52 😵

L'égalité ou rien !

12 mois de 30 jours ...

+ 5 jours épagomènes ("supplémentaires")
"sansculottides" tous les ans
+1 jour "sextile" tous les 6 ou 7 ans.

Bien d'autres ... mais **impossible de faire parfait**.

BDX I/O

Solution (calendriers) : les bonnes pratiques de code

> Temporal … learned from the many failures and dead bodies that came before it. And it had lots of good implementations to look at: Joda Time, Chrono, etc. – news.ycombinator.com

- ajouter une **durée "exacte"** : un nombre fixe de secondes → `timedelta(days=1)`
- ajouter une **durée "relative"** : « +1 mois », « +1 an », longueur qui dépend de la date de départ
  - une API qui accepte la réalité des calendriers : `relativedelta`, `Temporal.PlainDate.add`, Java `plusMonths`, SQL `interval '1 month'`
  - jamais de `replace(month= … )`, ni d'arithmétique à la main
- tester : 28 et 29 février, le 31, 1900 / 2000 / 2100 … et en property-based

BDX I/O

Solution (calendriers) : en pratique

Python : plante

```python
d = date(2026, 10, 31)
d.replace(month=11)  # ValueError: day 31 must be in range 1
d + relativedelta(months=1)  # 2026-11-30
```

JavaScript : déborde en silence

```javascript
const d = new Date(2026, 9, 31)  // mois de 0 à 11
d.setMonth(10)  // "Tue Dec 01 2026", donc month=11 !!
```

Go : déborde en silence

```go
d := time.Date(2026, 10, 31, 0, 0, 0, 0, time.UTC)
d.AddDate(0, 1, 0)  // 2026-12-01
```

Temporal : écrête, ou rejette sur demande

```javascript
const d = Temporal.PlainDate.from("2026-10-31")
d.add({ months: 1 })  // 2026-11-30
d.add({ months: 1 }, { overflow: "reject" })  // RangeError
```

Un résultat faux **silencieux** est pire qu'une erreur.

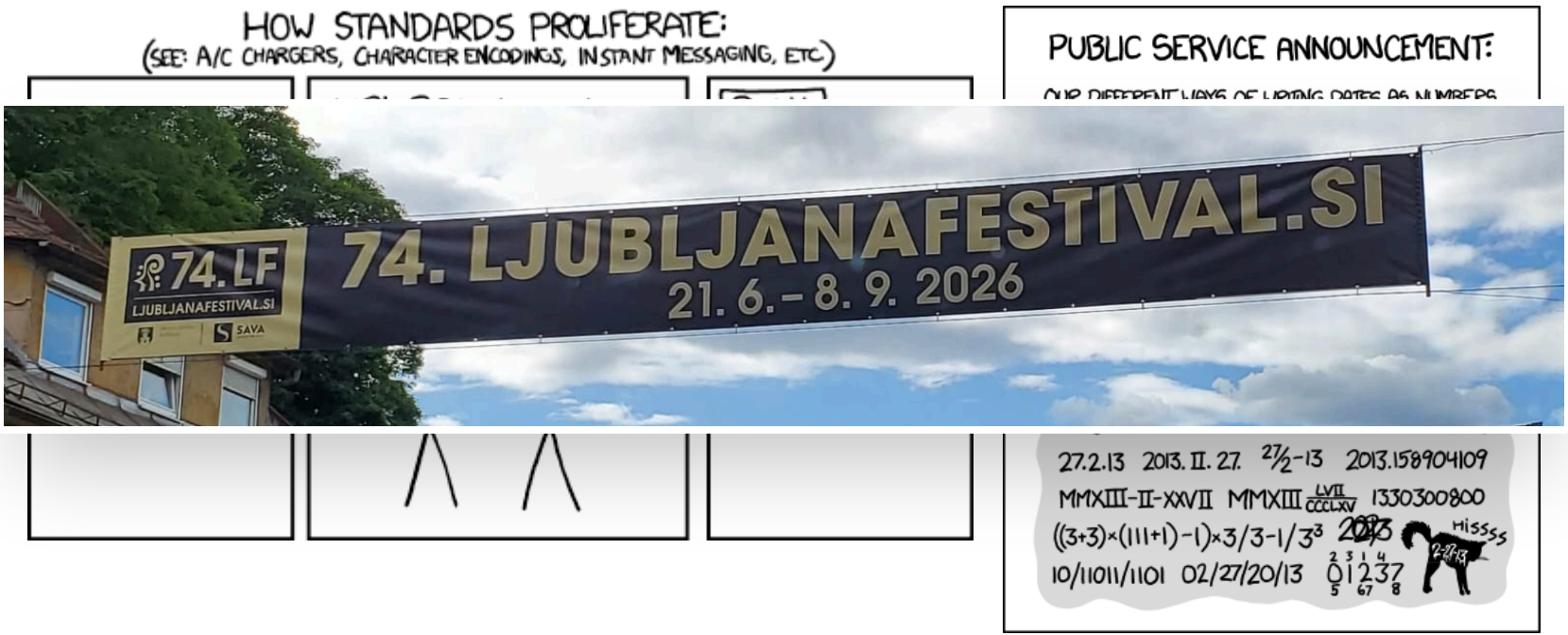
« +1 mois » d'une échéance, « +1 an » un 29 février : **règle métier explicite**, dernier jour du mois ou jour écrêté ?

BDX I/O

Quiz sur les formats !

1. Quel jour et mois désignent `10/11/2026` ❓
   - 10 novembre (mois n°11), format US `MM-DD-YYYY`
   - 11 octobre (mois n°10), format Européen `DD-MM-YYYY`
   - aucun moyen de savoir sans contexte
2. L'instant `08:19:01` est-il dans le passé ou le futur ❓
   - dans le passé dans cette salle, car il est `09:19:01`
   - dans le futur à Rio de Janeiro, car il est `05:19:01`
   - aucun moyen de savoir sans contexte
3. Je dois me lever tôt ou pas pour une réunion à « 8 heures » ❓
   - AM = matin ("ante meridiem")
   - PM = aprèm ("post meridiem")
   - aucun moyen de savoir sans contexte
4. Quel jour on sera « le mois prochain » ❓
   - Dimanche 1er ? (premier jour)
   - Vendredi 27 ? (même jour dans 4 semaines ≈ 1 mois)
   - Lundi 30 ? (même ordinal, ou presque)

La joie des standards



BDX I/O



ISO-8601 : pas très accessible

- ISO 8601:1988
- ISO 8601:1988/COR 1:1991
- ISO 8601:2000
- ISO 8601:2004
- ISO 8601-1:2019 (basic rules)
- ISO 8601-2:2019 (extensions)
- ISO 8601-1:2019/Amd 1:2022
- ISO 8601-2:2019/Amd 1:2025

> Only informative sections of standards are publicly available. To view the full content, you will need to purchase the standard by clicking on the "Buy" button. – iso.org

CHF 181,00 ↔️ EUR 199,15

ISO-8601 : de nombreux concepts

- années : `YYYY`
  - sachant que l'année `0000` n'existe pas dans le calendrier grégorien
  - `YYY` est autorisé pour désigner une décade,
- dates calendaires : `YYYY-MM-DD` ou `YYYY-MM` ou s")
- semaines de l'année : `YYYY-Www` ou `YYYYWww` (a
  - la première de l'année contient le premier jeu
  - la semaine 01 de 2027 commencera le 04 ja
- jours de semaine : `YYYY-Www-D` ou `YYYYWwwD` (a
- dates ordinales : `YYY-DDD` ou `YYYDDD` (avec `DD`
- "times" (heures) : `Thh:mm:ss.sss` (ou sans `.sss` ss.sss , ou `.mmm` , ou ...)
  - tellement de cas !
  - pas de limite au nombre de chiffres pour la pa ıe"
- offset : `Z` (prononcé "zulu") ou `±hh:mm` (ou sar
  - mais `-00` est interdit
  - depuis 2019 on peut ajouter `:ss` ou bien `:s`
- le `T` est obligatoire entre la partie "date" et la p
- durées : `PnYnMnDTnHnMnS` ou `PnW` ou `P<date>T<time>`
- intervalles : `<start>/<end>` ou `<start>/<duration>` ou `<duration>/<end>` ou juste `<duration>`
- répétitions : `Rn/<interval>` ou `R/<interval>`
- "Extended Date/Time Format" : incertitude, intervalles ouverts, exposants et chiffres significatifs, groupes de mois, dates multiples, ...

Current date and time expressed according to ISO 8601 [refresh]

Date in UTC	2026-10-29
Time in UTC	23:21:20Z T232120Z
Date and time in UTC	2026-10-29T23:21:20Z 20261029T232120Z
Date and time with the offset	2026-10-29T11:21:20-12:00 UTC−12:00
	2026-10-29T23:21:20+00:00 UTC+00:00
	2026-10-30T11:21:20+12:00 UTC+12:00
Week	2026-W44
Week with weekday	2026-W44-4
Ordinal date	2026-302

BDX I/O



RFC 3339

- "Date and Time on the Internet: Timestamps"
- moins de versions, changements plus mineurs
- définitif depuis 2002
- accessible gratuitement
- censé être un "profil" de l'ISO-8601 (version 1988 et 2000)
- proche, mais différent de ISO-8601 :
  - divergence : offset `-00` signifie "unknown"
  - divergence : un espace peut séparer la partie "date" de celle "time"
  - obligation : timezone offset
  - obligation : années sur 4 chiffres
- dans les faits, souvent confondus



Autres formats "standards"

- ISO/AWI 8601-1 : en cours de développement
- RFC 9557 : en cours de développement
- Unix timestamps : secondes écoulées depuis le 1er Janvier 1970 ("Epoch"), hors secondes intercalaires
  - `1794212342`
- Java timestamps : **milli**secondes écoulées depuis l'Epoch Unix
  - `1794212342895`
- RFC (2)822 ("mail date") : format "human-readable"
  - `Mon, 09 Nov 2026 09:19:02 +0100`
- RFC 7231 ("http date") : basé sur RFC 2822, mais toujours en GMT
  - `Mon, 09 Nov 2026 08:19:02 GMT`
- CNS 7648 : adaptation taïwanaise d'ISO 8601 au calendrier ROC (Grégorien - 1911)
- ...

Problèmes du temps

- Non-unicité : DST
- Non-monotonicité : DST
- Non-continuité : DST
- Non-homogénéité : DST, leap years
- Non-constance : DST, leap years
- Non-standardized : dates and time formats

Solution (formats) : échanger des dates

Contre-exemple

```
{
  "date": "10/11/2026",
  "created": "30/10/26 9h",
  "timeout": 30
}
```

Bon exemple

```
{
  "date": "2026-11-10",
  "created_at": "2026-10-30T08:00:00Z",
  "timeout_seconds": 30
}
```

- **toujours** un format standard : ISO 8601, ou plutôt son profil **RFC 3339**
- ⚠️ les langages **mélangent les deux noms** :
  - Python `isoformat()` : « ISO 8601 » (mais sans fuseau si le datetime est naïf)
  - JS `toISOString()` : « ISO 8601 simplifié » (lu par `Date.parse`, écrit toujours `Z`)
  - Go `time.RFC3339` : strict
  - Java `Instant.parse()` : « ISO 8601 » (fuseau obligatoire)
  - Rust (chrono) `to_rfc3339()` : écrit `+00:00` (pas `Z`), lit aussi avec un espace
- **pas de "format maison"**, ni à l'écriture, ni à la lecture
- l'**unité dans le nom** du champ : `timeout_seconds`, `duration_ms`, `size_bytes`

BDX I/O

Les petits détails ...

C'est quoi "GMT" ?

C'est quoi "+0200" ?

C'est quoi "Z" ?

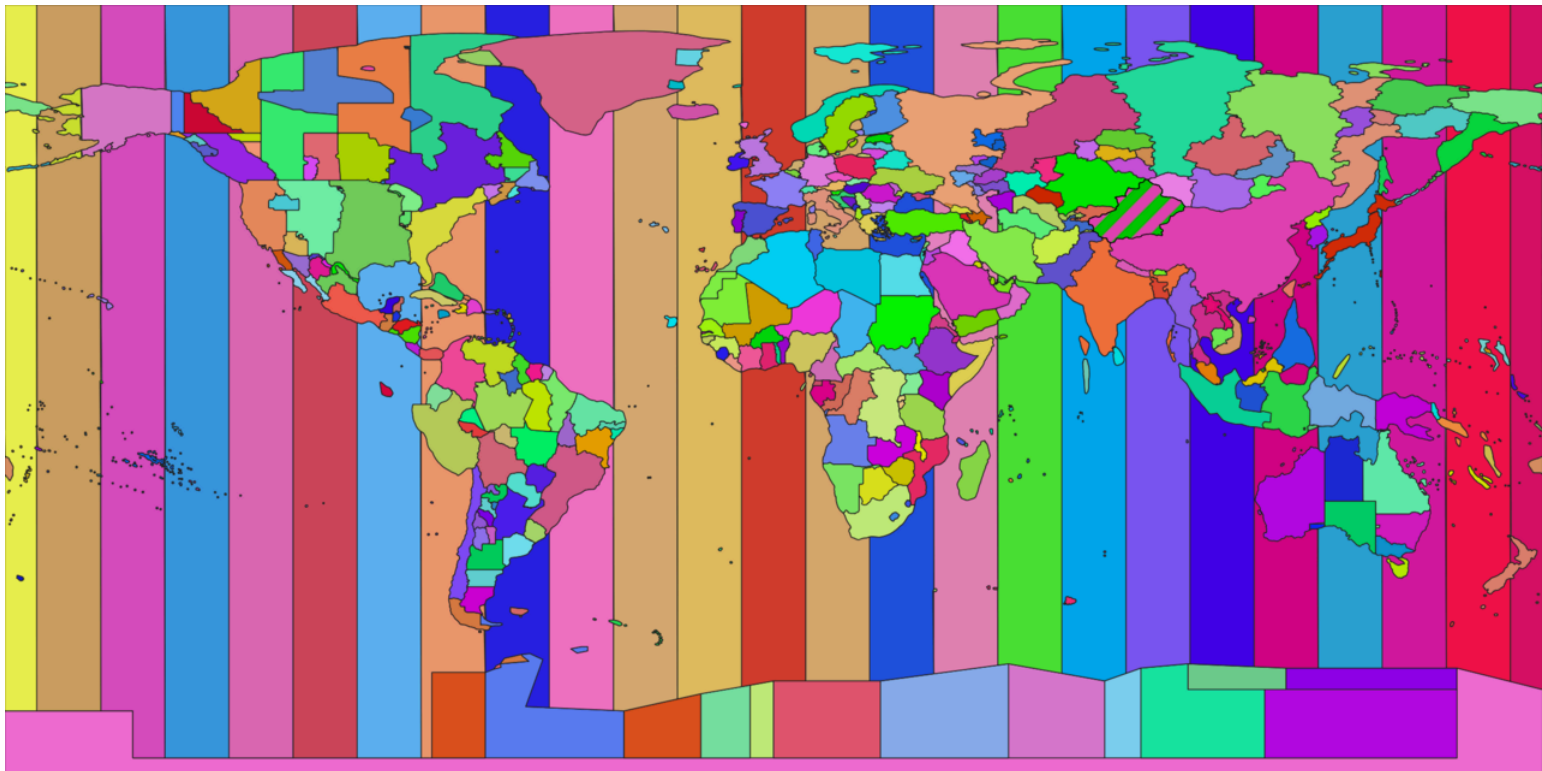


L’histoire des timezones

Pour faire (très) court :

- avant, le temps était très local
  - on suivait principalement le soleil
  - chaque ville avait son heure
  - on voyageait lentement
- puis les trains ont tout chamboulé
  - (les accidents mortels ont contribué)
- on a décidé d’unifier le temps, progressivement
- (je passe l’histoire chaotique pour chaque pays)
- aujourd’hui, le monde entier est découpé en timezones clairement définies

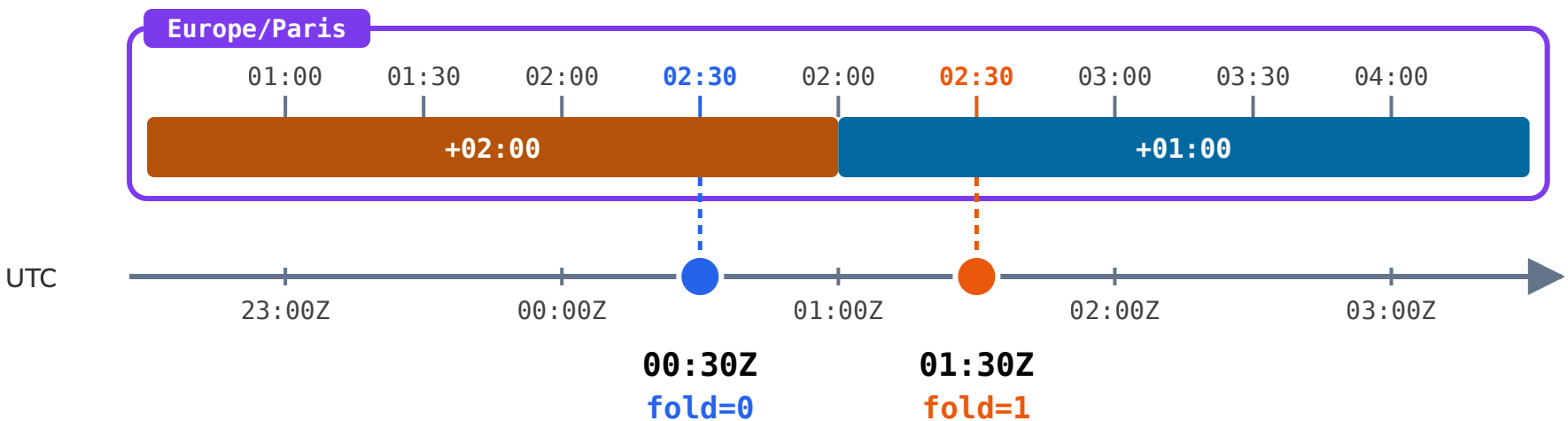
Tellement de timezones !



Offset, zone, naïf

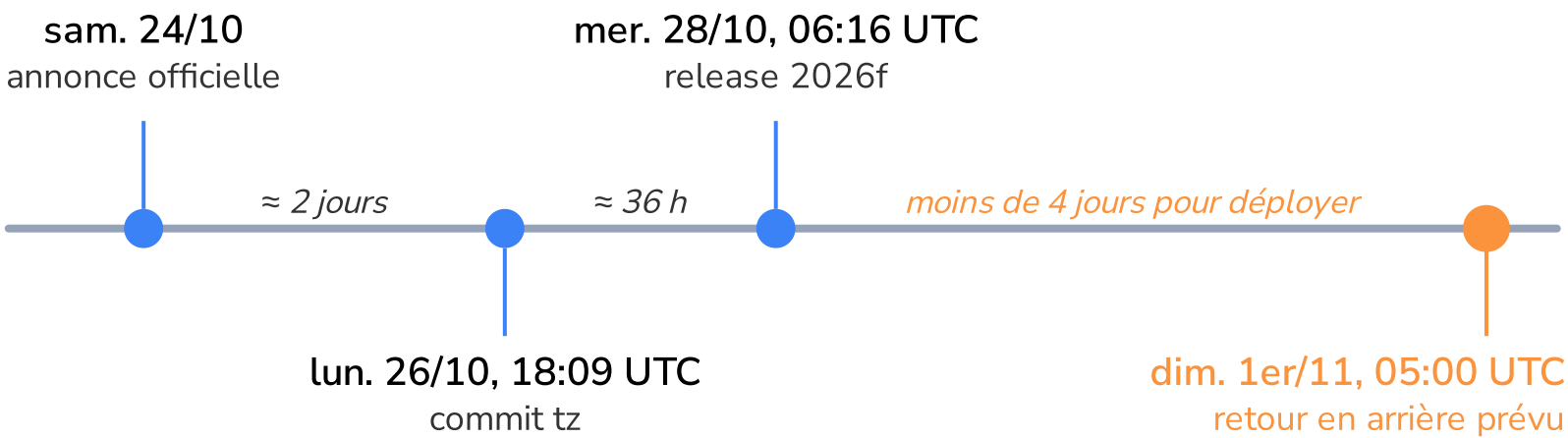
1 Offset | 2 Zone | **3 Naïf / zoned**

`2026-10-25 02:30` `Europe/Paris`

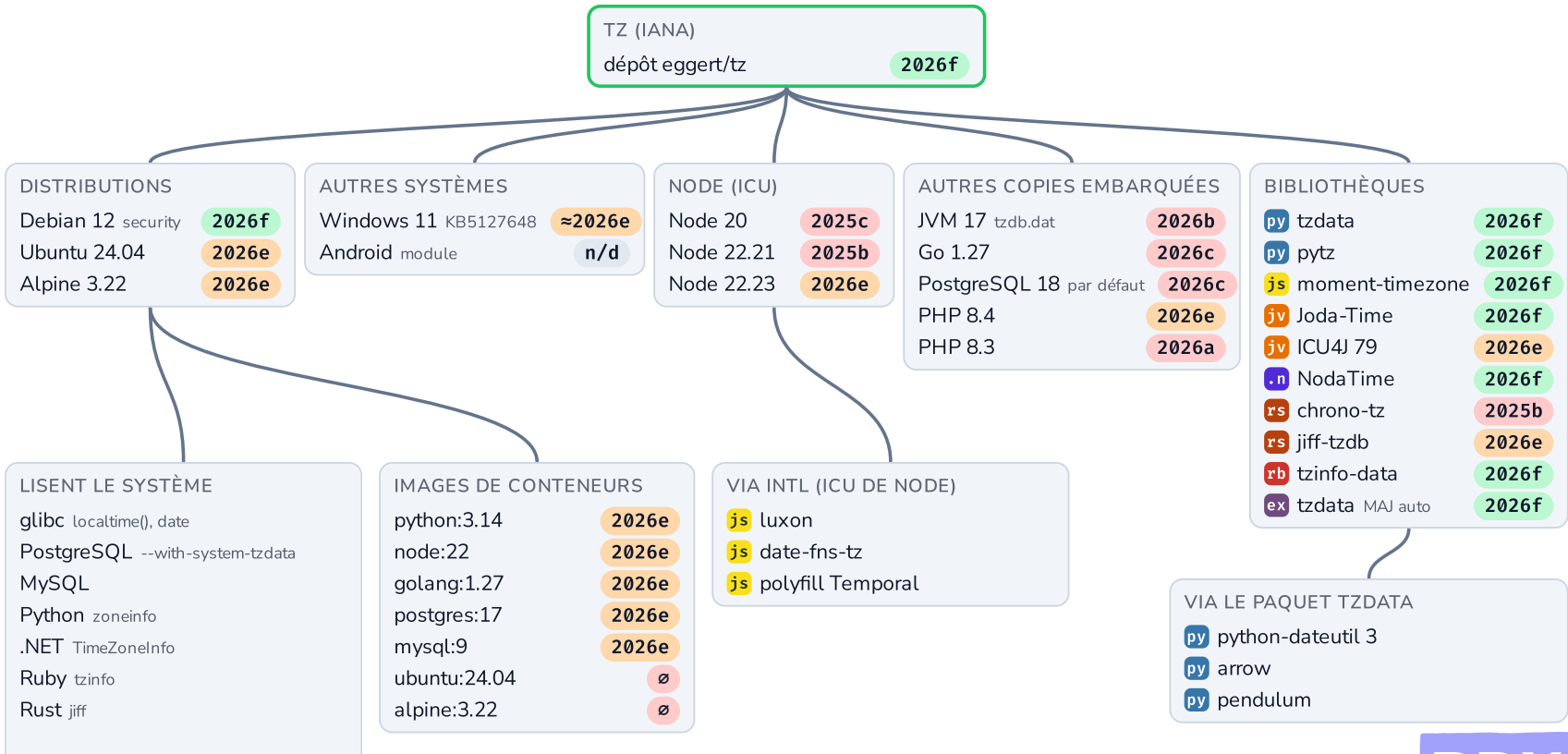


BDX I/O

Un sacré bazar !

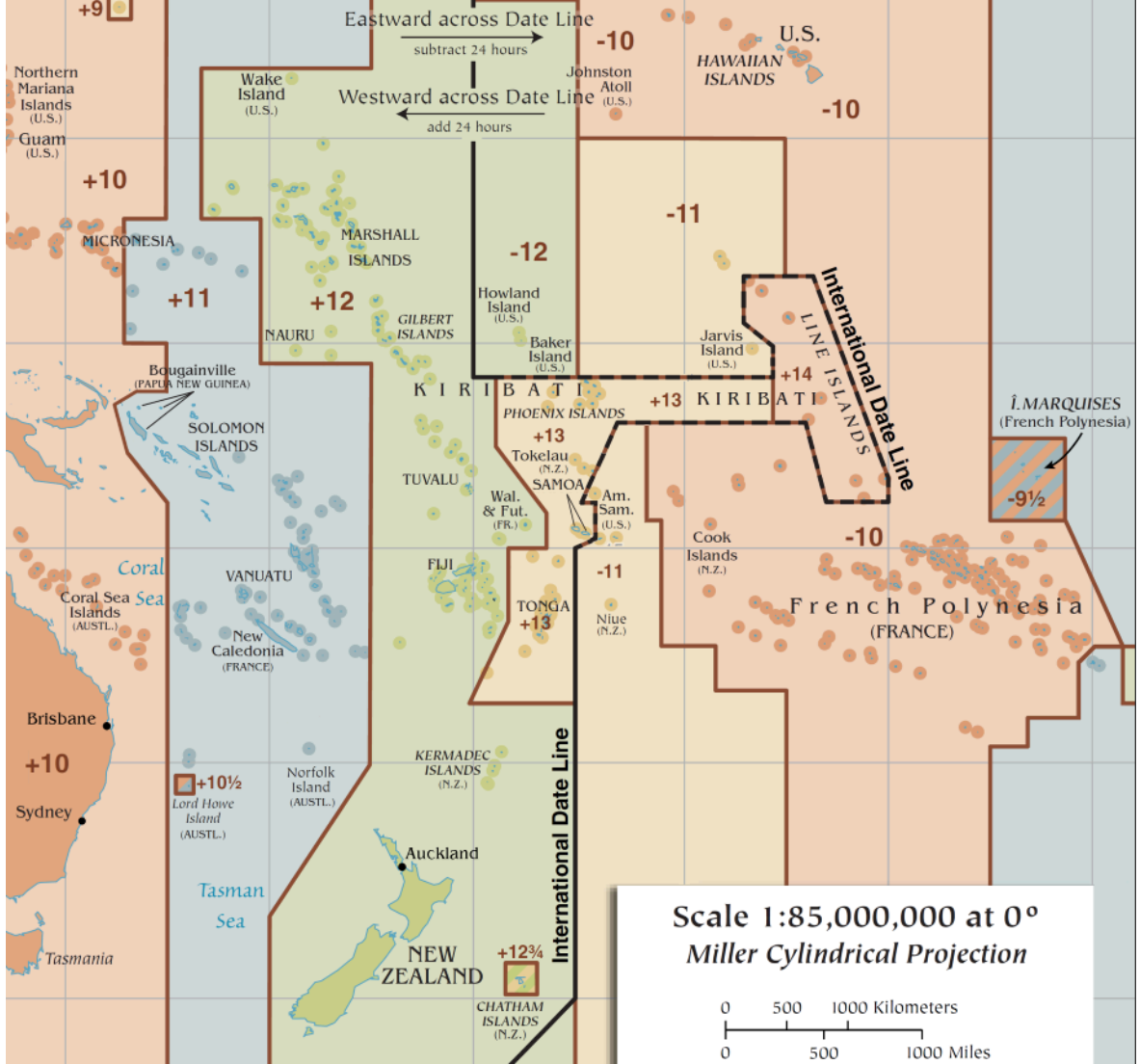


Qui a une copie de tzdata ?



Quel est le tzdata de ce que je déploie ?

BDX I/O


Eastward across Date Line
subtract 24 hours
Westward across Date Line
add 24 hours
International Date Line
International Date Line
LINE ISLANDS
+9
-10
U.S.
HAWAIIAN ISLANDS
Northern Mariana Islands (U.S.)
Guam (U.S.)
Wake Island (U.S.)
Johnston Atoll (U.S.)
+10
-10
MICRONESIA
MARSHALL ISLANDS
-11
-12
+11
+12
GILBERT ISLANDS
NAURU
Howland Island (U.S.)
Baker Island (U.S.)
Jarvis Island (U.S.)
+14
+13
KIRIBATI
PHOENIX ISLANDS
Bougainville (PAPUA NEW GUINEA)
SOLOMON ISLANDS
+13
Tokelau (N.Z.)
TUVALU
SAMOA
Wal. & Fut. (FR.)
Am. Sam. (U.S.)
Î. MARQUISES (French Polynesia)
-9½
Cook Islands (N.Z.)
-10
-11
FIJI
VANUATU
Coral Sea
Coral Sea Islands (AUSTL.)
TONGA
+13
Niue (N.Z.)
New Caledonia (FRANCE)
French Polynesia
(FRANCE)
Brisbane
+10
Sydney
+10½
Lord Howe Island (AUSTL.)
Norfolk Island (AUSTL.)
KERMADEC ISLANDS (N.Z.)
Auckland
Tasman Sea
Tasmania
NEW ZEALAND
+12¾
CHATHAM ISLANDS (N.Z.)
Scale 1:85,000,000 at 0°
Miller Cylindrical Projection
0 500 1000 Kilometers
0 500 1000 Miles

BDX I/O

Problèmes du temps

- Non-unicité : DST, timezones
- Non-monotonicité : DST, timezones
- Non-continuité : DST, timezones
- Non-homogénéité : DST, leap years, timezones
- Non-constance : DST, leap years, timezones
- Non-standardized : dates and time formats, timezones

Solution (timezones) : stocker et conserver

- saisie utilisateur : datetime dans le bon fuseau (zoned)
  - obligatoire, jamais de datetime "naïf" !
- convertir en 2 valeurs distinctes : datetime en UTC + fuseau voulu
  - (noter la version de tzdata utilisée pour le calcul)
- faire toutes les opérations sur la valeur UTC
- à la présentation, reconvertir la valeur UTC dans le fuseau voulu

Solution (timezones) : l'intention versus l'offset

- l'intention : « 10h à Mexico City » = une heure + une zone
- un offset (`-05:00`) est un résultat calculé, pas une intention
- une zone stocke un lieu plutôt qu'une valeur (stabilité !)
- l'UTC est un cache dérivé : le garder, mais le recalculer quand tzdata change
  - recalculer les événements postérieurs à la date d'effet de la règle (+ valeurs dérivées)
  - métier : décider que faire des événements déplacés ou déjà déclenchés à la mauvaise heure
- échanger :
  - RFC 9557 depuis 2024 (`2027-03-03T09:00+01:00[Europe/Paris]`), support parcellaire
  - sinon l'émuler avec : « RFC 3339 / ISO-8601 / … » UTC + string de la zone

BDX I/O

Solution (timezones) : maintenir tzdata

- inventorier : où tourne mon code, et d'où vient sa tzdata
- mettre à jour : releases fréquentes et délai court
  - reconstruire les images,
  - redémarrer les OS ou les services
  - faire monter de version les paquets
  - ...

Une date, ce n’est pas un instant

SKIP

```html
<input type="date" id="input" />
```

```js
const value = input.value
const instant = new Date(value)
instant.toISOString()
const fmt = { dateStyle: 'long', timeStyle: 'short' }
instant.toLocaleString('fr-FR', {
  ...fmt, timeZone: 'Europe/Paris',
})
instant.toLocaleString('fr-FR', {
  ...fmt, timeZone: 'America/New_York',
})
```

DÉMO

11/09/2026

VALEUR BRUTE

`"2026-11-09"`

une date : ni heure, ni fuseau

INSTANT

`2026-11-09T00:00:00.000Z`

`Date` en fait un instant : heure (00:00) et fuseau (UTC) inventés

PARIS

`9 novembre 2026 à 01:00`

NEW YORK

`8 novembre 2026 à 19:00`

lu dans un autre fuseau, la date change

I/O

09:19:02

Quand tzdata change : des questions à trancher

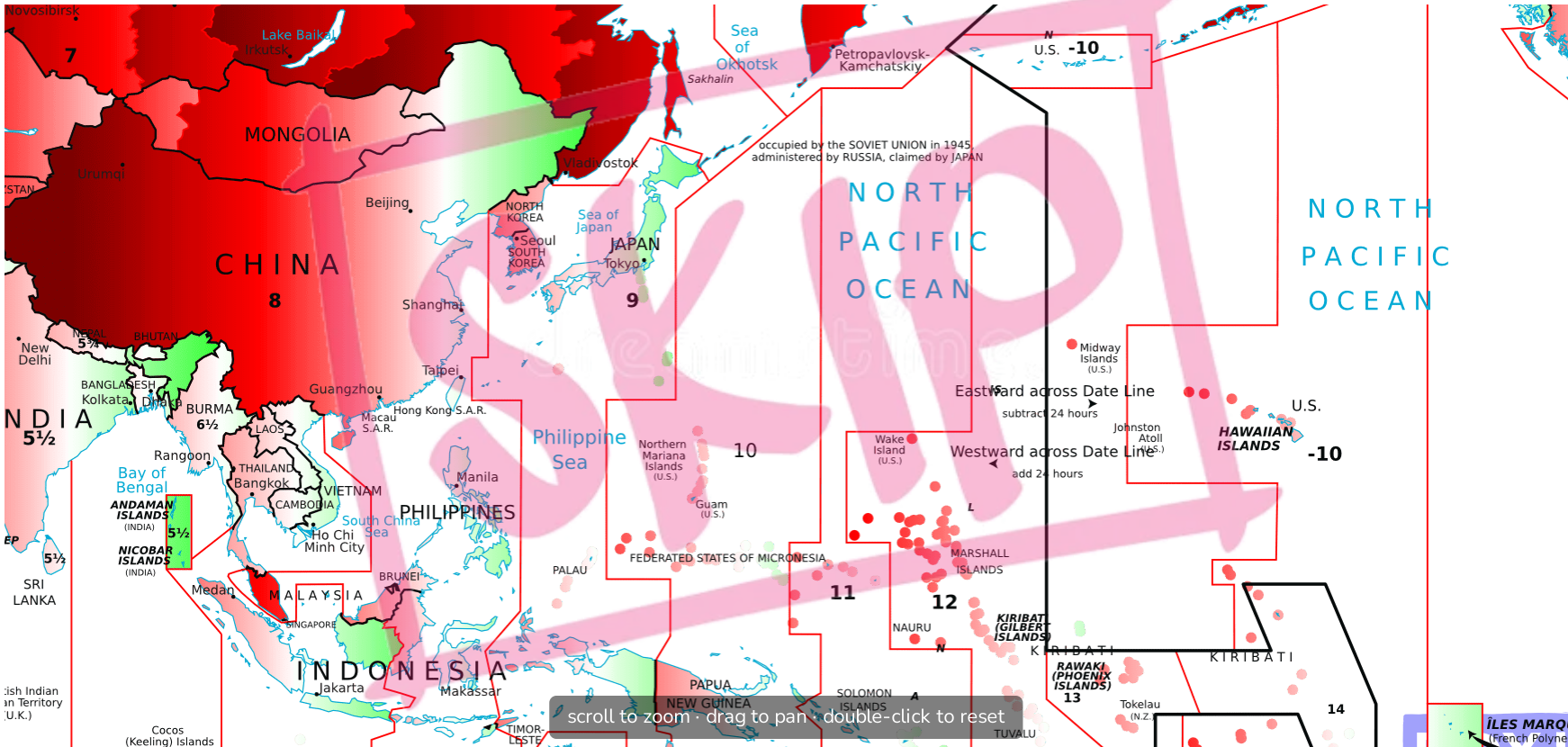
- Intention (heure + zone) : E_{ancien} et $E_{nouveau}$ sont l'UTC calculé avec l'ancienne et la nouvelle tzdata, U l'instant de la mise à jour



SKIP

- fait observé (log, paiement) : l'UTC est le fait, on ne le recalcule jamais
- une heure locale peut devenir inexistante ou ambiguë : Python choisit `fold=0` sans rien dire
- mise à jour avant la date d'effet : seul le premier cas existe

Petit bonus : décalage solaire



BDX I/O

Sans transition, les secondes !

- 09:19:02
- rien de plus simple : une toutes les secondes
- wait ...
- la seconde c'est l'unité SI de référence du temps
- > la durée d'un nombre entier d'oscillations (9 192 631 770 exactement) liées à la fréquence de transition hyperfine de l'atome de césium 133 – wikipedia.org
- mesuré à partir d'horloges "atomiques"
- on obtient le TAI : Temps Atomique International
- ➡️ quelle différence avec UTC ?

Le retour de la cosmologie !

- il faut d'abord parler de UT(1), et donc de UT0 ...
- à la base il y avait "GMT", différent de "GMT" (UTC+0)
- utilisé par les astronomes : chaque jour va de midi à midi, pour bosser la nuit
- transition à un calendrier minuit-minuit :

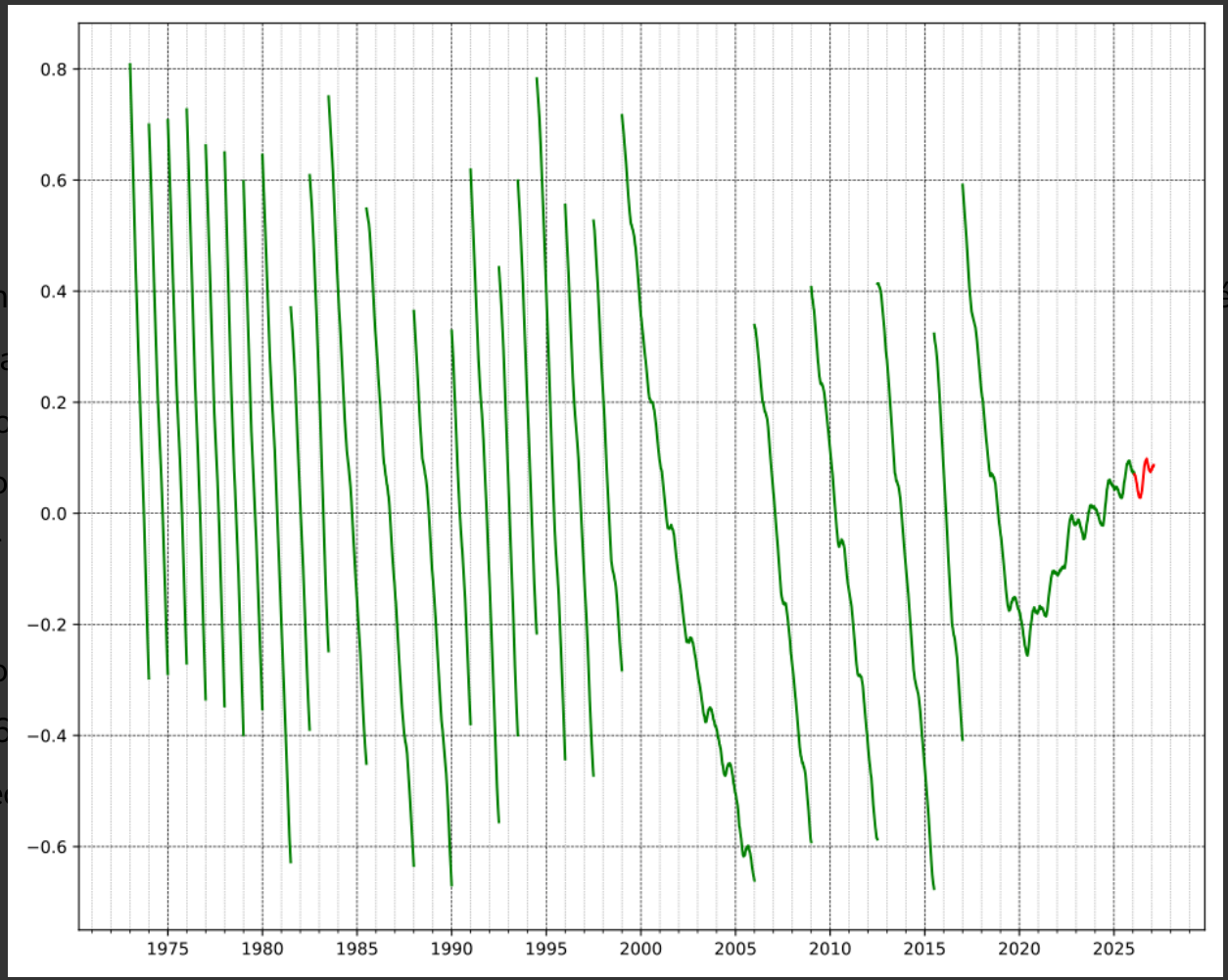
  Midi 31/12/1925 des astronomes devint minuit 31/12/1925
- renommé en UT
- devenu UT0 quand on a introduit UT1
- (UT1 prend en compte une correction du mouvement polaire)
- (je passe sur UT2)
- UT1 sert à calculer UTC

SKIP

Et

- un
- ma
  (fo
- do
- or
- et
- do
- 86
- se

éniths



BDX I/O

Problèmes du temps

- Non-unicité : DST, timezones, leap seconds
- Non-monotonicité : DST, timezones, leap seconds
- Non-continuité : DST, timezones, leap seconds
- Non-homogénéité : DST, leap years, timezones, leap seconds
- Non-constance : DST, leap years, timezones, leap seconds
- Non-standardized : dates and time formats, leap seconds

Solution (leap seconds) : la fin

Breaking news : 15 octobre 2026, CGPM, Résolution 5 : « On the technical actions needed to ensure the continuity of UTC »

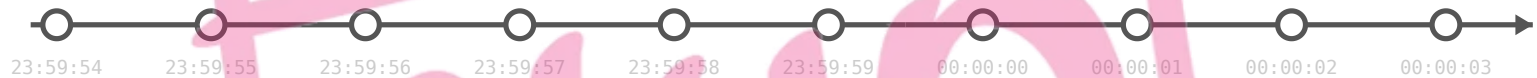
- UTC continu à partir du **20 mai 2027** : plus de seconde intercalaire !
- $UT1 - UTC$ pourra dériver jusqu'à **1 heure** (au lieu de 0,9 s)
- Dernière seconde intercalaire en date : 31 décembre 2016.
- désormais $TAI - UTC = 37s$



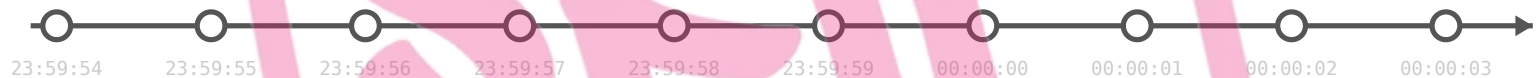
Et comment ça ~~fonctionne~~ fonctionnait ?

Ca dépend !

1. Ajout d'une seconde :



2. Smearing d'une seconde :

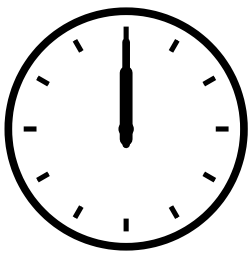


3. Répétition d'une seconde :

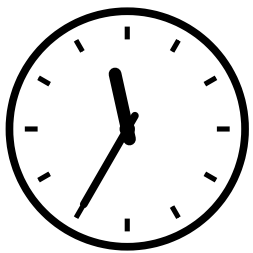


BDX I/O

Deux horloges ne sont jamais d’accord



12:00
machine A



11:35
machine B

1 an plus tard : **25 min** d'écart

- des logs dans le désordre d'une machine à l'autre
- un certificat TLS « pas encore valide » ou « expiré »
- un commit ou une release antidaté
- l'heure qui saute d'un coup, parfois en arrière !

➡️ qui remet les pendules à l'heure ?

Comment être toujours à l'heure ?

- ~~appeler "l'horloge parlante"~~ (ah non, plus depuis 2022)
- avoir sa propre horloge atomique (💸)
- avoir une montre mécanique ou au quartz
- calendrier solaire
- se brancher à un serveur NTP ("network time protocol")

  OS, FAI, Français, ...
- se brancher à un serveur PTP ("precise time protocol")
- monter son propre serveur (stratum)
- écouter la fréquence radio adéquate (SFTS : Standard frequency and time signal service)
- écouter les signaux GPS
- ...

SKIP

Démo : interroger un serveur NTP

Interroger un vrai serveur | Simuler (hors ligne)

capture du 2026-10-30 04:20:01

Résultat

décalage (offset)	−19.7 ms
délai (delay)	66.9 ms
stratum	2
refid	237.17.204.95
serveur	fr.pool.ntp.org 217.154.21.219

En-tête du paquet

leap indicator	0 pas d'alerte	version	4
mode	4 serveur	poll	0 (2^0 = 1.0 s)
precision	-25 (2^-25 = 3.0e-8 s)	root delay	8.87 ms
root dispersion	0.06 ms	refid (brut)	ed11cc5f IPv4 du serveur amont (ou hash)

Horodatages (UTC)

reference	2026-10-30T03:19:24.112885Z
T1 envoi client	2026-10-30T03:19:38.759766Z
T2 réception serveur	2026-10-30T03:19:38.773499Z
T3 envoi serveur	2026-10-30T03:19:38.773534Z
T4 réception client	2026-10-30T03:19:38.826662Z

BDX I/O

D’où vient l’heure de ma machine ?



- **stratum 0** : horloges de référence (atomique, GPS)
- **stratum 1** : serveurs branchés directement dessus
- **stratum 2** : synchronisés sur un stratum 1
- **stratum 3** : synchronisés sur un stratum 2, …
- **ma machine** : stratum 2 ou 3,
  à quelques ms de l’UTC (si tout va bien)

```
$ timedatectl timesync-status
       Server: 185.125.190.58 (ntp.ubuntu.com)
      Stratum: 2
       Offset: -34.612ms
        Delay: 47.362ms
       Jitter: 27.628ms
```

```
$ chronyc tracking
Stratum         : 3
System time     : 0.049011644 seconds fast of NTP time
Last offset     : -0.000111728 seconds
RMS offset      : 0.012802946 seconds
Frequency       : 4.796 ppm fast
Skew            : 6.584 ppm
```

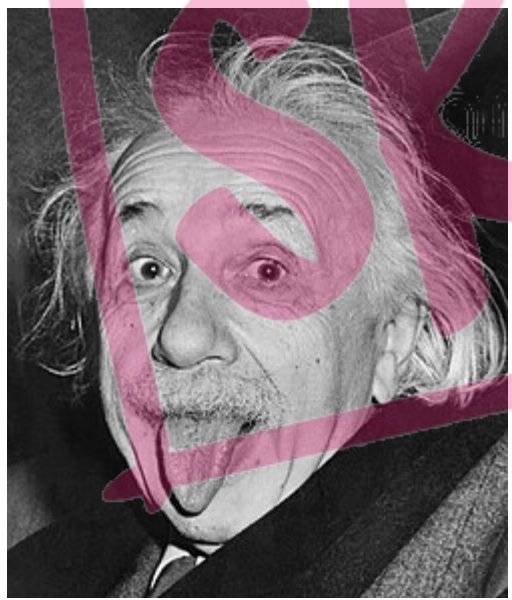
- alerter sur seuil (en ms)
- supervision Prometheus :
  `node_timex_offset_seconds` (node_exporter)
- ou via chrony :
  `chrony_tracking_system_time_seconds`
  (chrony_exporter)

Problèmes du temps

- Non-unicité : DST, timezones, leap seconds, clock drift
- Non-monotonicité : DST, timezones, leap seconds, clock drift
- Non-continuité : DST, timezones, leap seconds, clock drift
- Non-homogénéité : DST, leap years, timezones, leap seconds, clock drift
- Non-constance : DST, leap years, timezones, leap seconds, clock drift
- Non-standardized : dates and time formats, leap seconds

Le GPS c’est compliqué !

- ce sont des horloges atomiques en orbite autour de la terre
- elles signalent leur position orbitale, et leur "GPS Time" :

  nombre de secondes atomiques depuis le 6 Janvier, 1980, 00:00:00 UTC
- atomiques et pas UTC, donc il y a un décalage entre les deux !
- et il s’agit de quadrianguler la position du récepteur dans l’espace/temps 😛
- 

SKIP

Tout est relatif !

- la relativité restreinte : référentiels inertiels (sans gravité)
  - espace-temps
  - paradoxe des jumeaux
  - satellites GPS ou ISS
  - Interstellar (spoiler !)
- la relativité générale : référentiels en accélération (gravité)
  - espace-temps courbé
  - lignes droites hyperboliques dans l'espace
  - vitesse constante dans l'espace-temps
  - satellites GPS (45 µs/jour) ou dans l'ISS
  - Interstellar (spoiler !)
- voyage dans le temps :
  - uniquement vers le futur, plus ou moins vite que les autres
  - Oleg Kononenko (1111 jours) : a veilli 0.02+ secondes de moins

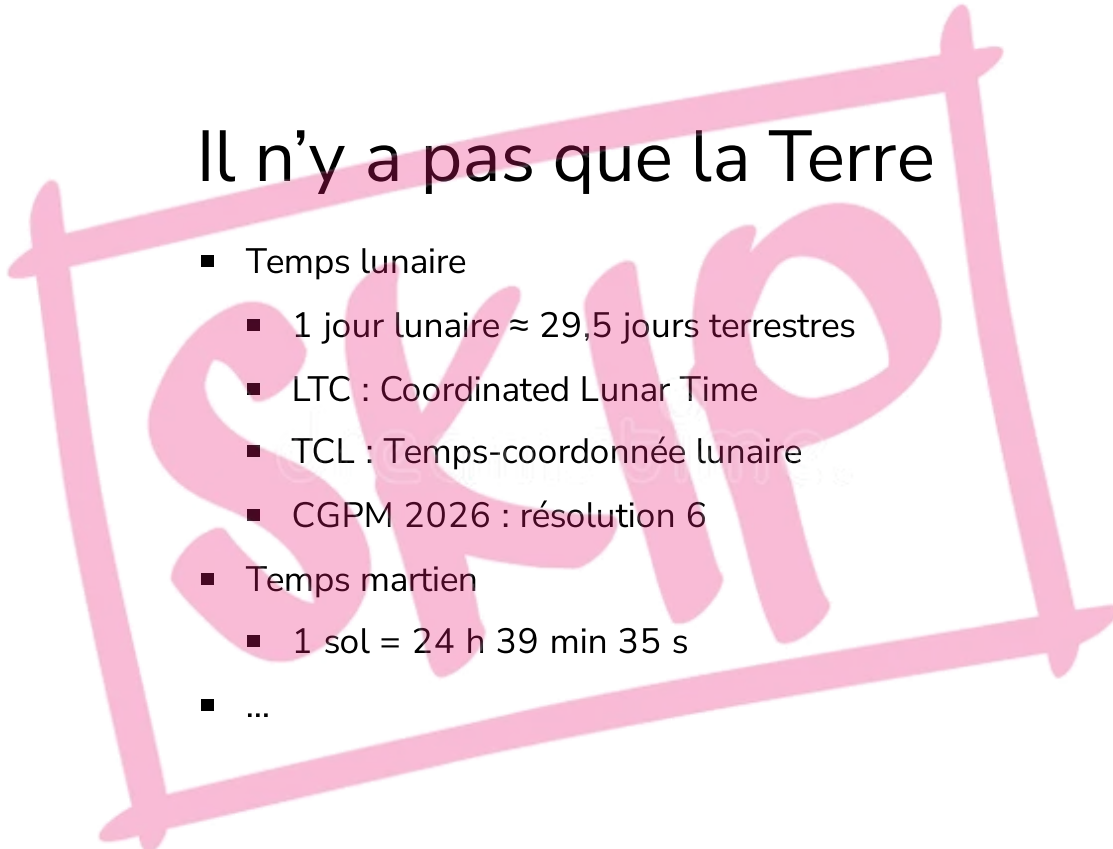
SKIP

Problèmes du temps

- Non-unicité : DST, timezones, leap seconds, clock drift
- Non-monotonicité : DST, timezones, leap seconds, clock drift
- Non-continuité : DST, timezones, leap seconds, clock drift
- Non-homogénéité : DST, leap years, timezones, leap seconds, clock drift, general and special relativity
- Non-constance : DST, leap years, timezones, leap seconds, clock drift, general and special relativity
- Non-standardized : dates and time formats, leap seconds

Il n’y a pas que la Terre

- Temps lunaire
  - 1 jour lunaire ≈ 29,5 jours terrestres
  - LTC : Coordinated Lunar Time
  - TCL : Temps-coordonnée lunaire
  - CGPM 2026 : résolution 6
- Temps martien
  - 1 sol = 24 h 39 min 35 s
- …



Les problèmes continuent



- Pourquoi il y a 2 dates de début du Ramadan en 2026 ?
- le mois commence avec l'apparition dans le ciel du premier quartier de lune
- l'observation peut donc différer d'un jour ou deux selon la longitude d'observation
- ▶ Détails des difficultés, d'après Wikipedia
- donc désaccords entre pays et communautés
- contribué aux recherches astronomiques par les scientifiques du monde arabe
- est-ce que l'observatoire d'Arabie Saoudite a plus raison que les autres ?
- la Grande Mosquée de Paris se base sur une méthode traditionnelle (mi-observation, mi-scientifique)
- alors que le Conseil français du culte musulman préfère une méthode seulement scientifique

Choix politiques

- Arrêter les secondes intercalaires : **décidé** le 15 octobre 2026 (CGPM)
- Arrêter les DST dans l'UE : consultation en 2018 (84 % pour), voté au Parlement en 2019, **bloqué au Conseil** depuis fin 2019, puis éclipsé par le Covid-19
- Changer les règles d'un pays (**surtout avec un préavis très court**) :
  - Égypte (2016f) : **2 jours**
  - Mexique (2022f) : **2 jours**
  - Colombie-Britannique (2026b)
  - Maroc (2026c)
  - Alberta (2026c)
  - Territoires du Nord-Ouest (2026d)
  - Manitoba (2026e)
  - Cuba (2026f) : **5 jours**

Conclusion

- déterminer le temps dans le passé n’est pas simple
- déterminer si deux instants passés sont simultanés n’est pas simple
- déterminer dans combien de temps se produira un instant futur est impossible
- en fait, le temps est toujours très délicat à manipuler
- complexité essentielle méconnue
- complexité accidentelle méconnue
- les IA font ces erreurs aussi, dans le code et les tests
- si vous voulez vous entrainer : FalsehoodsAboutTime.com
  - Falsehoods programmers believe about time (34)
  - More falsehoods programmers believe about time; “wisdom of the crowd” edition (79)

BDX I/O

Pour survivre dans la jungle

Problèmes

- non-unicité
- non-monotonicité
- non-continuité
- non-homogénéité
- non-constance
- non-standardisation

Vocabulaire

- instant, UTC, offset
- naïf contre zoned
- date seule
- durée exacte contre durée relative

Stocker et échanger

- stocker l'intention
- types de base de données
- unités et epoch
- RFC 3339 / RFC 9557

Tester et détecter

- injecter l'horloge dans le code
- contrôler l'horloge dans les tests
- tester avec un autre `TZ`
- tester les cas limites

Exploitation

- mettre à jour tzdata
- surveiller la dérive d'horloge
- TZ=UTC

Secondes

- secondes intercalaires et smearing

BDX I/O

Remerciements

- Meetup QA de Grenoble (et Rachel en particulier)
- Claude Code + Sonnet
- Slidev.js
- Petter Salminen - It’s All About Time ⏰ - What developers need to know about time
- Le personnel du bar "Le Not a Beer"

- Photo of a croissant by personalgraphic.com on Unsplash
- Images des évolutions des DST par Wikipedia : DST_Countries_Map.png
- Noël en Australie, © myflag.com.au
- Image du calendrier Symmetry454 par Wikipedia : page Symmetry454
- Image du calendrier républicain par Wikimedia : Deuxième partie d'un calendrier de 1801-1802, avec les correspondances grégoriennes, orné de signes allégoriques, G.29855.jpg
- Image de multiples calendriers par Wikipedia : Gregorian calendar
- Image des timezones par Wikipedia : Timezone-boundary-builder release 2023d.png
- Image des timezones de l'Arizona par : travelness.com
- Image des timezones autour de l'International date line par Wikipedia : International Date Line.png
- Image du décalage entre UTC et heure solaire par 24timezones.com
- Image du décalage entre UT1 et UTC par Wikipedia : Leapsecond.ut1-utc.svg
- Photo d'Albert Einstein par Wikimedia : Albert Einstein sticks his tongue 1951.jpg
- Capture des exemples ISO 8601 par Wikipedia : ISO 8601
- Capture du bug de l'Apple Watch par iPhoneSoft : article du 26 octobre 2026
- Photo d'une affiche du Ljubljana Festival par Julien Lenormand, CC BY-SA 4.0

Ressources pour aller + loin

Culture

- Erik Naggum - The Long, Painful History of Time : très complet, surtout pour la vision historique
- Revues du monde - Le futur n'a pas toujours existé : vidéo qui démonte notre rapport au temps, très lié à notre époque/culture
- Scilabus - Le jetlag social, ce décalage que vous vivez chaque week-end

Timezones

- qntm - So You Want To Abolish Time Zones : courte fiction sur les problèmes introduits par l'abolition des timezones
- qntm - So You Want Continuous Time Zones : courte fiction sur les problèmes causés par la multiplication des timezones
- "Discussion" sur Hacker News des raisons de choisir une time zone ou une autre : en résumé, un "political move" vis-à-vis de l'administration
- Colin Eberhardt - Exploring 120 years of timezones : récit et visualisations de l'évolution des timezones au cours du temps
- Judah Levine - Everyday Time and Atomic Time: Part 4 : l'histoire de l'ancien GMT
- Computerphile - The Problem with Time & Timezones : vidéo un peu dramatique des problèmes de timezones
- The Supreme Guide to World Time Zones : très complet et avec de jolies images

Astronomie et astrophysique

- Launch Pad Astronomy - The Sky Part 1: Local Sky and Alt-Az / Horizon Coordinates : explication des coordonnées célestes Altitude-Azimuth + très jolie visualisation des mouvements célestes en 2nde partie
- FloatHeadPhysics - "I finally understood why time even exists! (My mind is blown)" : explication du "temps propre relativiste" plutôt claire
- Le Sens of Wonder - Le voyage dans le temps est possible : une réponse sérieuse (et décevante) à la question
- ScienceClic - Que verrait-on en tombant dans un Trou Noir ? : de magnifiques images pour comprendre les effets de la courbure de l'espace-temps
- ScienceEtonnante - Le paradoxe des jumeaux : rajeunir à la vitesse de la lumière ? : le célèbre paradoxe, expliqué assez clairement
- Matthias Kreier - Relative time for GPS satellites : des calculs détaillés pour comprendre les tailles d'effet

Tooling

- jsdate.wtf : le recueil des pièges de `Date` en JavaScript
- Astropy : bibliothèque Python pour l'astronomie, avec ses propres échelles de temps (UTC, TAI, TT, UT1...)
- Symfony Clock : `MockClock` pour figer l'horloge en PHP, basé sur PSR-20
- Temporal : propose un glossaire parfait, devenu standard
- Arie Bovenberg - Ten Python datetime pitfalls, and what libraries are (not) doing about it : des exemples de bugs causés par de mauvais outils, ayant mené à la création de `whenever`
- BurntSushi - about leap seconds : Why leap seconds wont be supported in jiff, and why they are complicated and not very useful, also some reading recommendations
- Open Dataset des Années bissextiles sur gouv.fr : la version officielle en France de si une année est bissextile ou pas

BDX I/O

Questions et slides

feedback sur Roti.express



code des slides sur GitHub

